1.INTRODUCTION

1.1 Statement of the Problem

Parkinson's Disease (PD) is a long-term neurological condition that mainly disrupts motor functions, leading to symptoms like shaking, muscle stiffness, slowed physical movements (bradykinesia), and problems with balance and posture.As the disease advances, these motor symptoms become more pronounced and can significantly impair the quality of life. Early and accurate diagnosis is crucial for initiating timely treatment, managing symptoms effectively, and slowing the progression of the disease. However, traditional diagnostic methods, such as clinical observation, neurological exams, and imaging techniques, are often subjective, expensive, and may not detect the disease in its early stages.One of the emerging and non-invasive techniques for the early diagnosis of Parkinson's Disease involves analyzing hand-drawn spirals. Drawing tasks, such as the Archimedean spiral, are commonly used in neurological assessments because they can reveal subtle motor dysfunctions. Patients with Parkinson's typically exhibit micrographia (abnormally small handwriting) and tremor patterns that distort the spiral shape. These distortions are difficult to quantify visually but can be effectively captured and analyzed using advanced computational methods. The rise of artificial intelligence and machine learning, especially in medical image processing, has sparked increasing interest in creating automated tools capable of identifying Parkinson's Disease through the analysis of spiral drawings. Machine learning models can learn intricate patterns and features from spiral images that are associated with Parkinsonian symptoms. These models can then be used to classify whether a spiral drawing belongs to a healthy individual or a patient with Parkinson's, with high accuracy and reliab

1.2 Objectives:

- To develop an automated system for Parkinson's disease detection using deep learning models.
- To utilize a publicly available Parkinson's drawing dataset comprising hand-drawn spirals and wave patterns.
- To implement and compare different CNN architectures including MobileNet, EfficientNet, and ResNet.
- To train the models using preprocessed and augmented image data to increase prediction robustness.
- To evaluate each model based on performance metrics like accuracy, precision, recall, F1-score, and confusion matrix.
- To integrate the best-performing model into a web-based interface using Flask or similar technologies.
- To provide a user-friendly system that allows healthcare workers or users to upload drawing images and get instant predictions.
- To reduce the dependency on traditional, high-cost medical diagnosis procedures.
- To create a scalable solution that can be improved in the future with larger datasets or real-world integration.

1.3 Scope

- The scope includes:
    - Dataset preparation and cleaning.
    - Image preprocessing and visualization.
    - Implementation of three deep learning models.
    - Model training, testing, and evaluation.
    - Comparison of model results.
    - Deployment of one model via a simple UI for demo purposes.

1.4 Methodology

- **Step 1: Dataset Acquisition**
    - A Parkinson's drawing dataset is used containing spiral and wave images from healthy and affected individuals.
    - Images are categorized into four classes: healthy_spiral, parkinson_spiral, healthy_wave, and parkinson_wave.
- **Step 2: Data Preprocessing**
    - Images are resized to match model input dimensions (e.g., 224x224).
    - Grayscale conversion, normalization, and augmentation are performed to improve model performance and generalization.
- **Step 3: Model Development**
    - Three different deep learning models are used: MobileNet, EfficientNet, and ResNet-50.
    - Pretrained models are fine-tuned using transfer learning techniques.

3. LITERATURE REVIEW

3.1. Existing and Proposed System

Clinical-Based Diagnosis:

- Parkinson's Disease is currently diagnosed through neurological assessments and physical examination by specialists.
- Symptoms such as tremors, stiffness, slowness of movement, and impaired balance are observed.

Imaging Techniques:

- Advanced imaging methods like MRI, PET, and DaTscan are sometimes used to support clinical diagnosis.
- These techniques are expensive, not widely accessible, and require specialized equipment.

3.1.1 Disadvantages:

- Data Quality Issues: Spiral drawings may vary due to external factors like stress, fatigue, or device sensitivity, affecting model accuracy.
- Limited Dataset Availability: High-quality, annotated spiral drawing datasets for Parkinson's detection are scarce, limiting model training.
- False Positives/Negatives: The model might misclassify healthy individuals or early-stage patients, leading to incorrect diagnosis.
- Device Dependency: Results can vary based on the input device (stylus, finger, tablet), affecting consistency.

3.2Proposed System

Problem Addressed:

- The proposed system is designed to detect Parkinson's Disease using deep learning models applied to hand-drawn spiral and wave images.
- It aims to build a fully automated, low-cost, and scalable tool that eliminates the dependency on manual diagnosis and imaging devices.

Key Features of the Proposed System:

- Utilizes MobileNet, EfficientNet, and ResNet deep learning architectures.
- Works on image classification of spiral and wave drawings collected from patients.
- Offers fast, real-time predictions using a trained model deployed in a web application.
- Requires no expensive medical equipment or wearable sensors.
- Provides accurate and objective results suitable for both clinical and home settings.

3.2.1Advantages

- Fully automatic — no doctor/specialist required for initial diagnosis.
- Can be deployed on web or mobile platforms.
- Low resource requirements — runs on any standard PC or laptop.
- Much cheaper than MRI or wearable-based solutions.
- Easy to operate — user simply uploads an image and receives instant results.
- Promotes early detection and awareness, especially in remote or rural areas.

Innovation and Uniqueness:

- Instead of relying on sensors or audio input, it uses a simple image of a spiral/wave, which can be easily captured using a phone or scanner.
- Multiple deep learning models are compared to select the most reliable one.

3.3 Feasibility Study

- The project is based on deep learning models (MobileNet, EfficientNet, and ResNet), which are proven architectures in image classification tasks.
- All models are implemented using Python with popular libraries such as TensorFlow, Keras, OpenCV, Matplotlib, and NumPy.
- The code was executed successfully using Kaggle Notebook/Google Colab, which provides free GPU access for training.
- Pretrained versions of these models are readily available via Keras Applications, supporting transfer learning.
- Spiral and wave images are simple to preprocess (resize, grayscale, normalize), making them suitable for CNN models.
- The Flask framework is used to develop the backend of the web application, which is lightweight and well-suited for model deployment.
- No specialized hardware (e.g., high-end GPUs or servers) is required to run the trained models for prediction — a basic PC or laptop is sufficient.
- The system is designed with ease-of-use in mind — users can simply upload an image and receive a disease prediction.

3.4 Tools and Technologies

Programming Language: Python

- Python is the primary language used in this project.
- It is widely supported in the machine learning and deep learning community.
- Features readable syntax, a vast number of libraries, and cross-platform compatibility.
- Suitable for prototyping, deployment, and web integration.
- Version Used: Python 3.7+

Libraries and Frameworks

TensorFlow & Keras

- Used for building and training deep learning models (MobileNet, EfficientNet, ResNet-50).
- Keras simplifies model design with a high-level API built on top of TensorFlow.

NumPy

- Supports numerical computations and multidimensional array operations.
- Used extensively for handling image matrices and normalizing pixel values.

Pandas

- Provides data manipulation tools for handling tabular data (e.g., file names, labels).
- Used for dataset organization and CSV file operations.

Matplotlib & Seaborn

- Visualization libraries used to plot training curves, accuracy graphs, and confusion matrices.
- Help in analyzing model performance and interpreting results.

OpenCV

- Used for reading, resizing, and preprocessing images.
- Handles tasks like grayscale conversion and image augmentation.

OS & Shutil

- Python built-in libraries used for:
    - Navigating directories.
    - Copying and organizing image files.

Deep Learning Models

MobileNet

- A lightweight CNN architecture optimized for mobile and embedded vision applications.
- Used in this project for faster training and real-time prediction performance..

EfficientNet

- A scalable and efficient model that balances width, depth, and resolution using compound scaling.
- Used to test performance under resource constraints while maintaining high accuracy.

ResNet-50

- A deep convolutional neural network with residual (skip) connections.
- Enables training of very deep models without vanishing gradient issues.

Development Platforms

Kaggle Notebook

- Used for model training and experimentation with GPU support.
- Enables seamless access to public datasets and free compute resources.

Dataset Source

Kaggle Dataset – Parkinson's Drawing Dataset

- Contains spiral and wave images from both healthy individuals and Parkinson's patients.
- Four image categories:

Web Application Framework

Flask

- A Pythn-based micro web framework used to deploy the best-performing model.
- Provides the backend logic for handling.

Interface Tools (Frontend)

HTML & CSS

- Used for creating a simple and clean front-end user interface.
- Users can upload an image and view results within a browser.

4. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

4.1 Users

1. Primary Users

- **General Public / Patients**
  - Purpose: To upload spiral/wave drawings and receive diagnostic predictions.
  - Technical Skill: Basic (can use a browser and upload an image).
- **Healthcare Professionals**
  - Purpose: Use the system as a support tool for early Parkinson's screening.
  - Technical Skill: Moderate (comfortable using digital tools).

2. Secondary Users

- **System Administrator**
  - Purpose: Maintain model, update datasets, manage backend server.
  - Technical Skill: Advanced (web, Python, Flask, TensorFlow).
- **ML Developers / Data Scientists**
  - Purpose: Retrain or update the deep learning model with new data.
  - Access Level: Development-level access (model code, dataset, training environment)

4.2 Functional Requirements

1. Image Upload Function

- Users can upload spiral or wave drawings (JPG, PNG).
- The uploaded image is validated for format and size.

2. Model Prediction

- The system loads the trained model (e.g., ResNet).
- Image is preprocessed (resize, grayscale, normalize).

3. Result Display

- Shows prediction result in readable format.
- Displays confidence level (if implemented).

4. Data Logging (Optional)

- Logs each prediction request (timestamp, result).
- Used for auditing or improving model accuracy.

5. User Interface

- Clean web interface for user interaction.
- Includes file upload, result view, and reset functionality.

6. Admin Functions

- Upload updated model files.
- View usage logs.

4.3 Non-Functional Requirements

1. Performance Requirements

- Response time for prediction: < 2 seconds per image.
- Model loading time: < 10 seconds (once per session).

2. Security Requirements

- Image uploads should be sanitized to prevent code injection.
- No user data is stored permanently.

3. Availability and Uptime

- System should be available 24/7 with minimal downtime.
- Deployment on a cloud server ensures remote access.

4. Maintainability

- Code is modular and well-documented.
- Models are stored separately for easy update.

5. Usability

- Web interface should be mobile and desktop compatible.
- Simple layout with clear instructions.

6. Scalability

- System can scale to support additional models or diseases.
- Backend allows integration of APIs and mobile apps in future.

7. Testability

- Each module (upload, preprocessing, prediction) is testable independently.
- Logs and return values can be verified during unit testing.

8. Compatibility

- Works on Windows, MacOS, Linux.
- Compatible with Chrome, Firefox, Edge browsers.

5. SYSTEM DESIGN

5.1 System Architecture

This diagram represents the entire Parkinson's Disease Detection System as a **single process** with its interactions with external entities. It shows how data flows in and out of the system without detailing internal operations.

Entities Involved

1. User (Patient/Healthcare Professional)

- Uploads a spiral or wave image.
- Receives prediction output from the system.

2. Parkinson Detection System (Main Process)

- Preprocesses image.
- Feeds image into the deep learning model.

3. Deep Learning Model

- Loaded internally (MobileNet, EfficientNet, or ResNet).
- Returns a predicted class label.

4. Web Interface

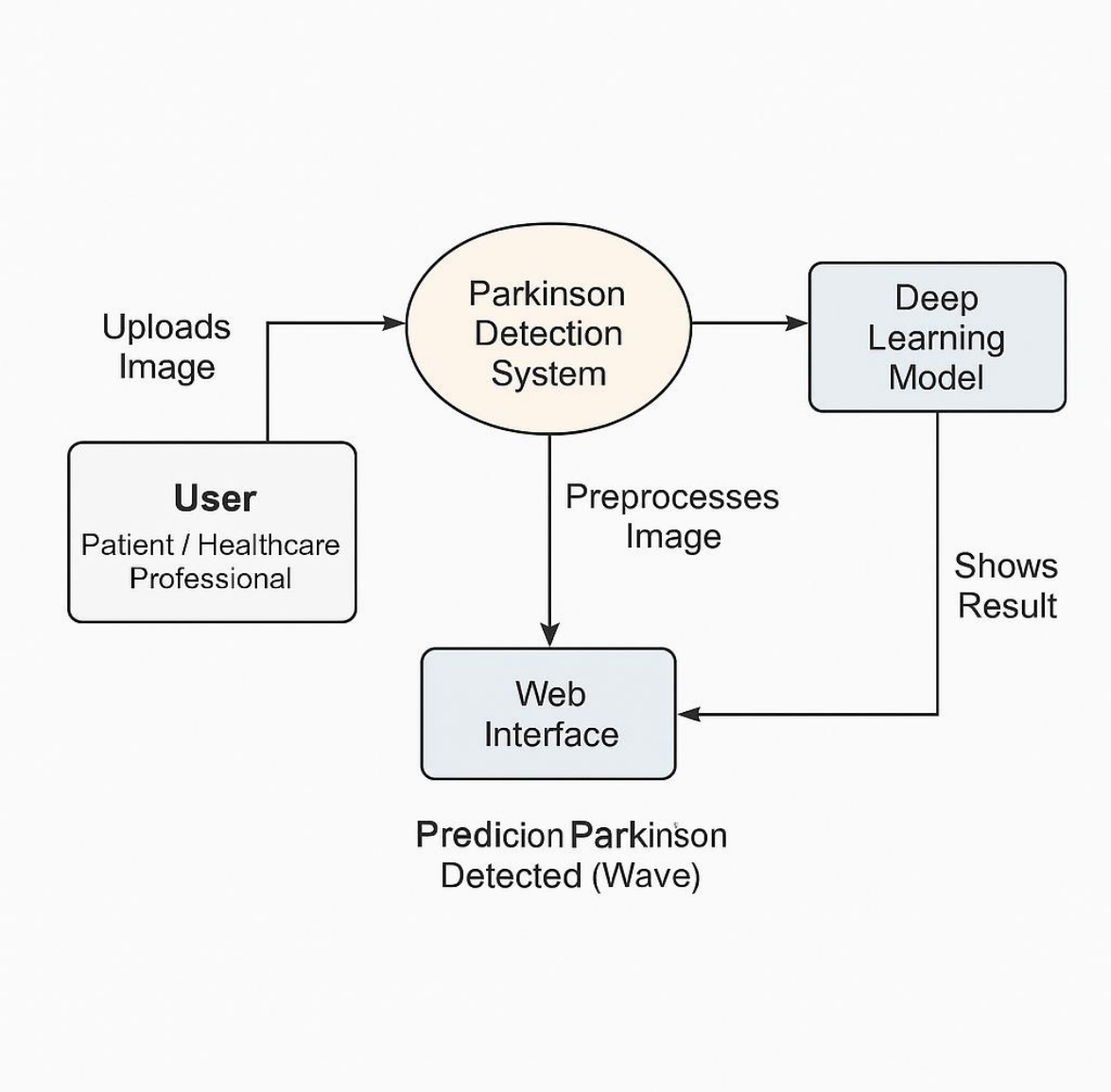
- Frontend where user interacts with the system.
- Displays upload panel and final result.

Data Flow Points

1. **User ➞ Uploads Image ➞ System**
2. **System ➞ Preprocesses Image ➞ DL Model**
3. **DL Model ➞ Returns Prediction ➞ System**
4. **System ➞ Shows Result ➞ User**

Explanation:

- The user does not interact with the internal model directly.
- The system handles image formatting and invokes the appropriate pre-trained model.
- The final output is sent back to the user for informational or diagnostic support.
- This DFD helps understand external communication without detailing processing steps.



5.2 Flow Diagram

The system follows a **layered architecture** consisting of the **User Interface**, **Backend Logic (Flask)**, **Preprocessing**, **Model Inference**, and **Offline Training and Storage** components.

1. User Interface (HTML, CSS)

- **Role:** This is the **frontend** where the user uploads their drawing (spiral or wave).
- **Technology:** HTML & CSS provide a basic interface.
- **Flow:** The image is submitted to the Flask server for further processing.

2. Flask Web Server (Backend Controller)

- **Role:** Acts as the **brain** of the system.
- Receives image upload from the user.
- Calls the preprocessing module.
- Passes the image to the deep learning model for prediction.
- Returns prediction back to the frontend.

3. Preprocessing Module

- **Role:** Prepares the uploaded image to be compatible with the trained model.
- Tasks include:
    - Resize (e.g., 224x224 px).
    - Grayscale conversion (if needed).
    - Normalization.
- Output is a model-ready image.

4. Model Inference (Deep Learning Model)

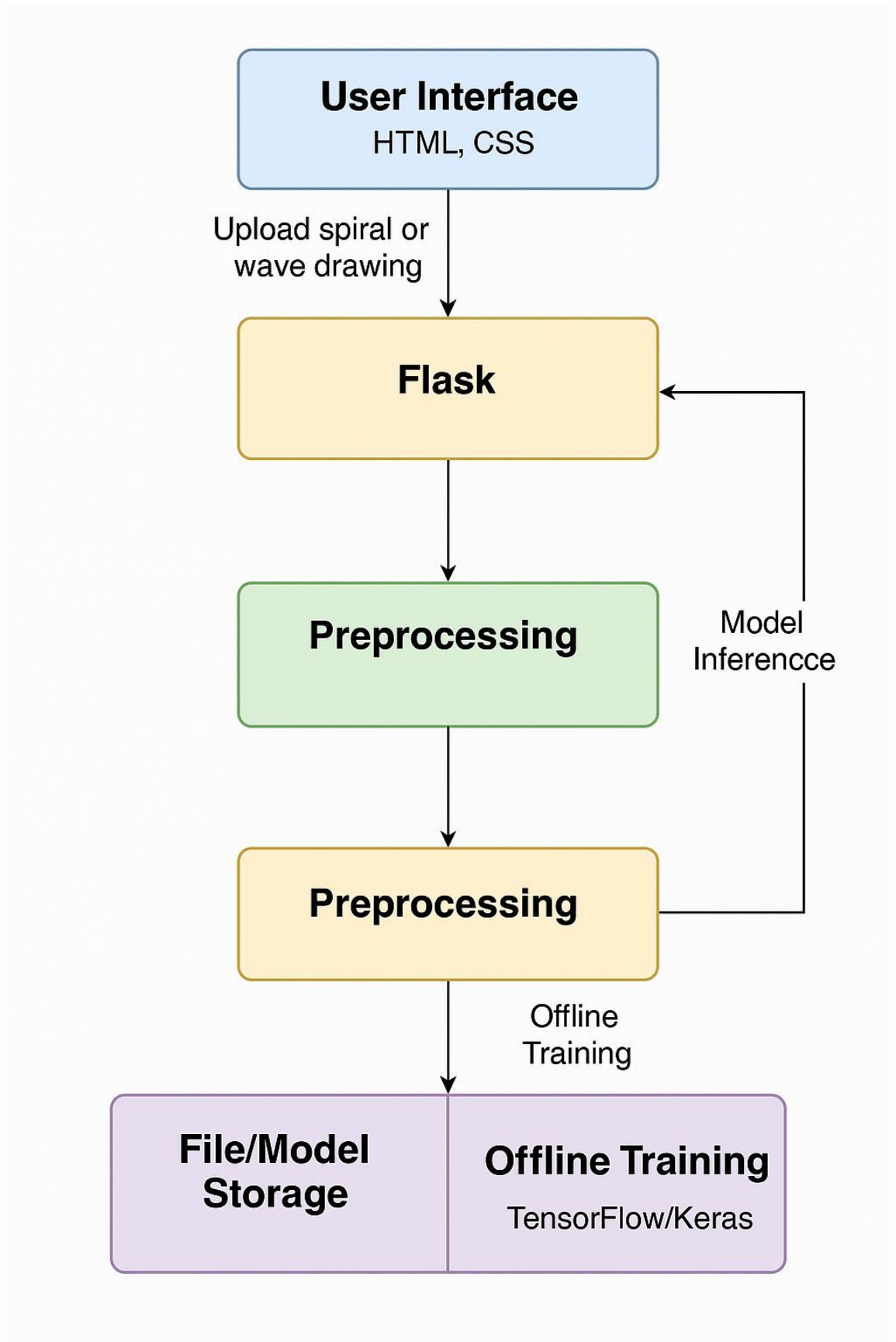
- **Role:** Performs classification using a pre-trained model like **MobileNet**, **EfficientNet**, or **ResNet-50**.
- Model is loaded using **Keras/TensorFlow**.
- Returns a prediction:
    - Parkinson Spiral
    - Healthy Spiral
    - Parkinson Wave
    - Healthy Wave

5. File/Model Storage

- Stores:
    - Uploaded images (temporarily).
    - Trained model files (.h5 format).
- Ensures separation of data and models for clean deployment.

6. Offline Training (TensorFlow/Keras)

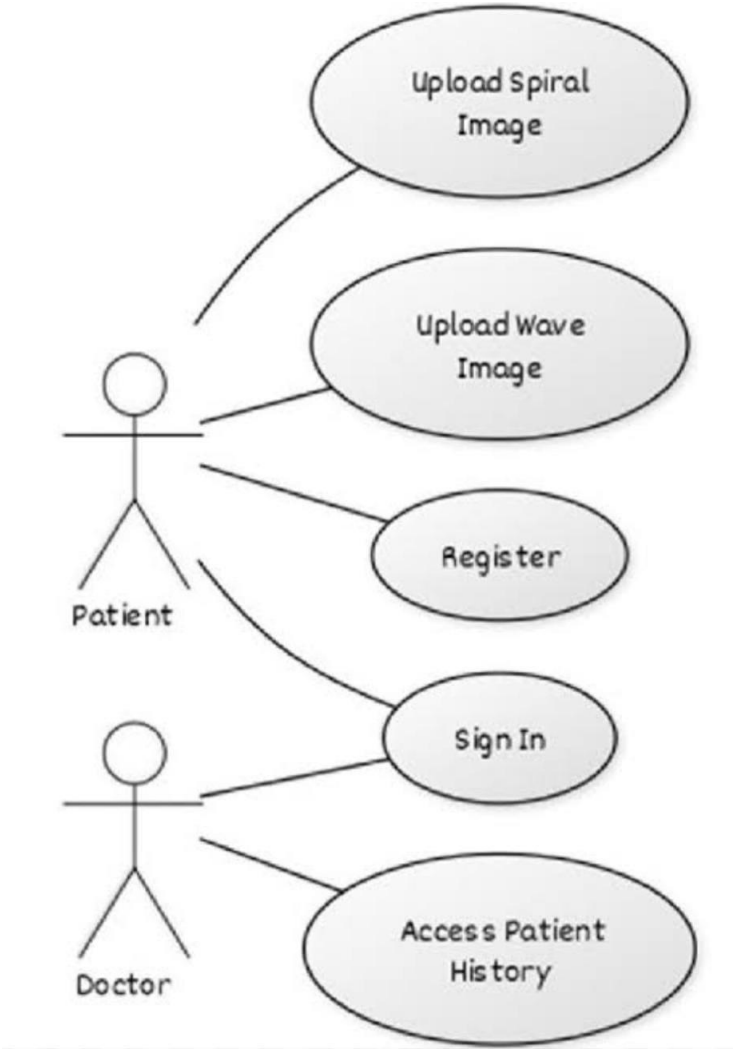
- Training is conducted using tools like **Google Colab** or **Kaggle**, where:
    - Datasets are loaded.
    - Models are trained.
    - Final models are exported as .h5 files.


User Interface
HTML, CSS
Upload spiral or wave drawing
Flask
Preprocessing
Model Inferencce
Preprocessing
Offline Training
File/Model Storage
Offline Training
TensorFlow/Keras

6.DETAILED DESIGN:

6.1Use Case Diagram

The **Use Case Diagram** represents the functional interaction between **users** and the **Parkinson's Disease Detection System**. It identifies the major use cases supported by the system and the types of actors interacting with it.



The Use Case Diagram represents the functional interaction between users and the Parkinson's Disease Detection System. It identifies the major use cases supported by the system and the types of actors interacting with it.

5.1 Purpose of Use Case Diagram

- Helps in understanding what the system does from an end-user perspective.
- Represents external users (actors) and their interactions (use cases) with the system.
- Useful during the requirement analysis and design phases.

5.2 Actors in the System

1. User (Patient or Healthcare Professional)

- Interacts with the frontend UI.
- Uploads spiral or wave images.
- Receives disease prediction as output.

2. Admin / Developer

- Manages the backend and model updates.
- Accesses logs or updates trained models.
- Performs retraining or testing.

5.3 Use Cases (Functions)

1. Upload Image

- User uploads a hand-drawn image (spiral or wave).
- System accepts JPG/PNG formats.

2. Preprocess Image

- System resizes, grayscales, and normalizes the image.
- Prepares the input for the deep learning model.

3. Generate Prediction

- System loads the pre-trained model (MobileNet, EfficientNet, or ResNet).
- Returns a class prediction (e.g., Parkinson Detected or Healthy).

4. View Prediction Result

- Prediction is shown to the user via the web interface.
- May include probability/confidence score.

5. Update Model (Admin only)

- Admin uploads a new model (.h5 format).
- System replaces the old model with the new one.

6. Access Logs (Admin only)

- Admin can view history of user uploads and predictions.
- Useful for system audit or retraining data collection.

7. Manage Dataset (Optional)

- Admin adds or removes labeled images from the training dataset.
- Supports model retraining for improved accuracy.

5.4 Description of Diagram Elements

Symbol	Meaning
Ellipses	Represent use cases (functions)
Stick Figures	Represent actors (user/admin)
Rectangle Box	System boundary
Arrows	Show interaction between actor and system

5.5 Use Case Scenarios

Use Case: Upload Image

- Actor: User
- Precondition: User opens the website.
- Action: Uploads spiral or wave image.
- Postcondition: Image is sent for preprocessing.

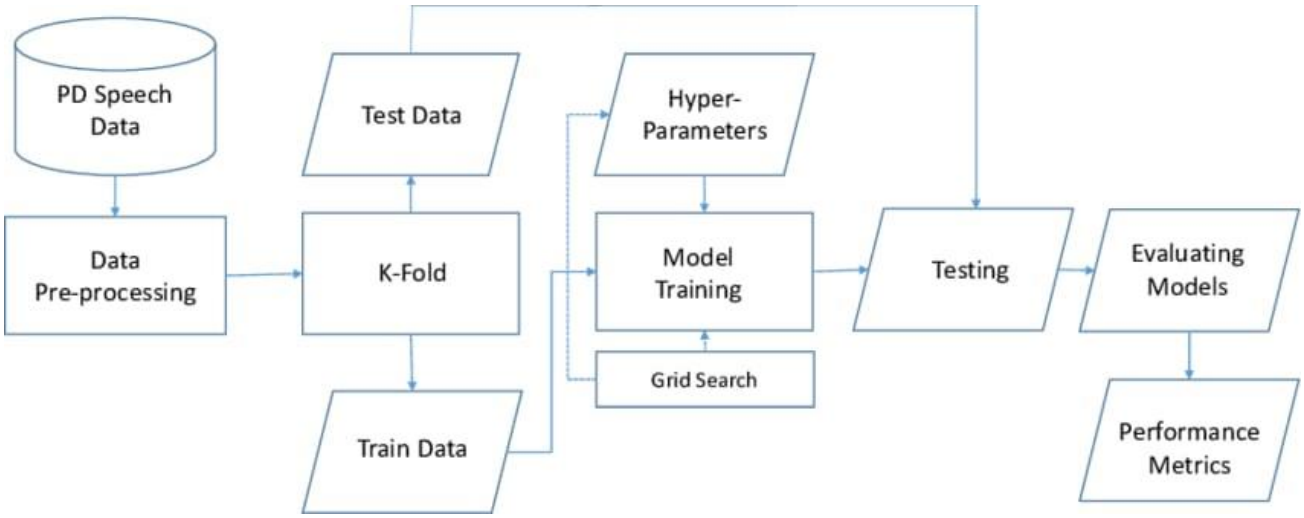
Use Case: Generate Prediction

- Actor: System
- Trigger: Image upload triggers model prediction.
- Outcome: System shows diagnosis (Parkinson/Healthy).
- Constraint: Model must be already loaded in memory.

Use Case: Update Model

- Actor: Admin
- Precondition: Admin is authenticated.
- Action: Uploads new model file.
- Postcondition: New model is loaded for future predictions.

6.2 Activity Diagram



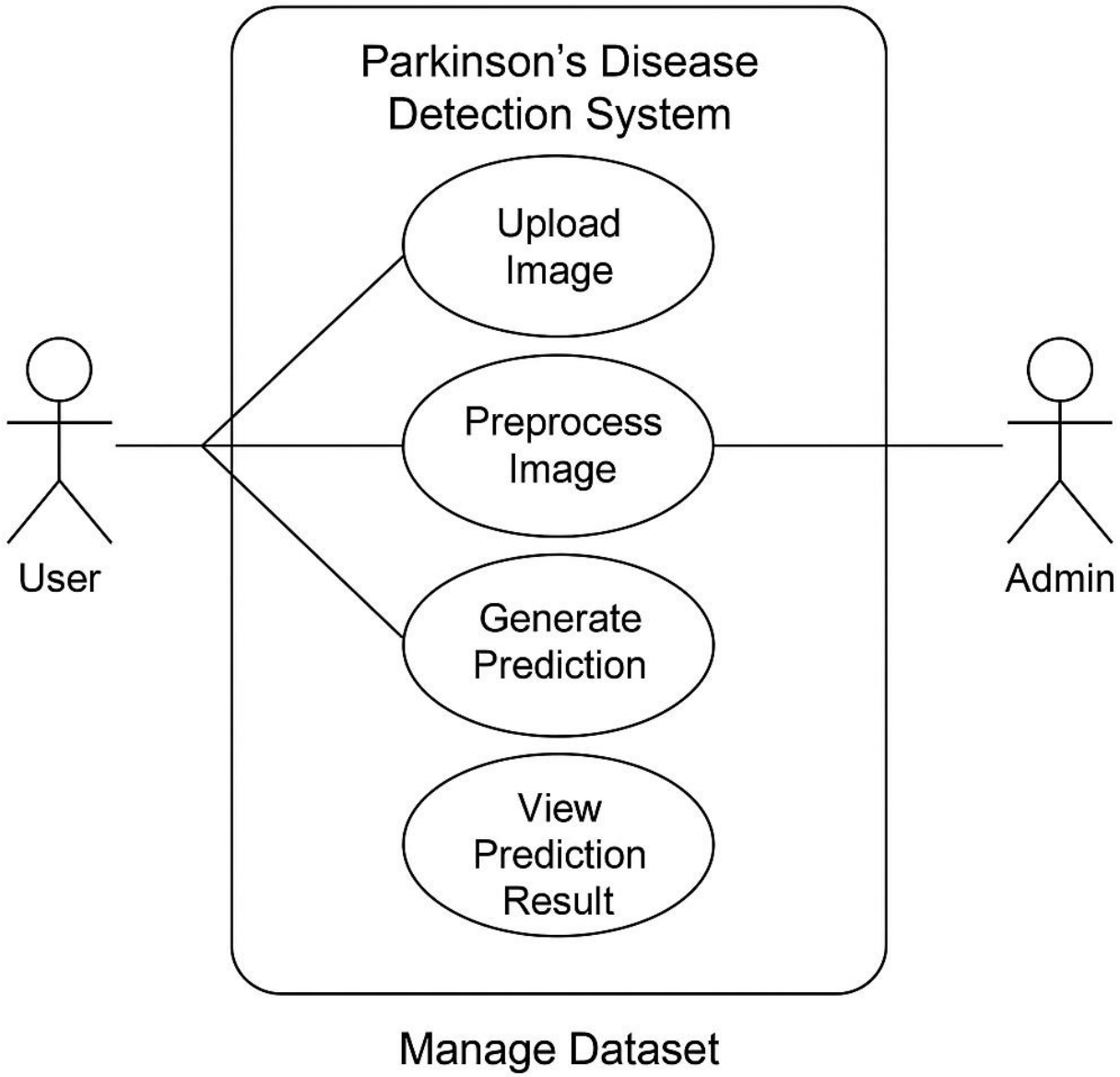
- The user (patient or doctor) accesses the Parkinson Detection System via a web browser.
- The web interface is served using Flask, built using HTML/CSS.
- This is the entry point of the system interaction.
- The user chooses a spiral or wave drawing image from their local device.
- Only image files (.jpg, .jpeg, .png) are allowed.
- This file will be used as input to detect signs of Parkinson's disease.
- The selected image is sent to the backend using a POST request.
- The Flask server receives the file and temporarily stores it.
- The image is now ready for backend processing.

- The Flask backend checks:
    - Is the uploaded file an image?
    - Is it in accepted format (.jpg, .png)?
    - Is it within the size limit (e.g., not a corrupted file or too large)?

.

- Image is resized (e.g., to 224x224 pixels).
- Color channels might be reduced (grayscale).
- Pixel values are normalized (scaled between 0 and 1).
- Ensures consistency with the model's training data.
- The cleaned image is fed into the trained model (MobileNet, EfficientNet, or ResNet).
- The model is already loaded in Flask using Keras or TensorFlow.
- The model performs **inference** using the uploaded image.
- Output is usually in the form of class probabilities.
- Example:
    - Class 0: Healthy Spiral – 12%
    - Class 1: Parkinson Spiral – 88%
- The highest probability class is chosen as the final result.
- Additional data like confidence level or model name used may also be included.
- The final diagnosis is shown in a readable format:
    - "Parkinson Detected (Wave Drawing)"
    - Or: "Healthy Spiral"
- Displayed in the web UI with proper formatting and optional disclaimer:
- The session is complete.
- The system may optionally:
    - Clear uploaded file
    - Log prediction (optional)
    - Offer user a "Try Again" option

6.3 Sequence Diagram



Sequence diagrams describe **how objects interact** in a particular scenario of a system. They show the **time-based flow of messages** between actors (users, admin) and system components (Flask server, model, etc.).

Purpose of Sequence Diagrams

- Shows the **order of operations** for specific functionalities.
- Helps visualize the **flow of logic** between frontend, backend, and model.

User Uploads Image and Receives Prediction

Actors:

- User (Patient/Doctor)
- Web Interface (HTML/Flask)
- Flask Backend
- Preprocessing Module
- Trained Deep Learning Model (ResNet/EfficientNet/MobileNet)

Scenario Steps:

1. **User** opens the web app.
2. **User** selects and uploads an image (spiral or wave).
3. **Frontend** sends the image to **Flask backend** via POST request.
4. **Flask** receives the image and passes it to the **Preprocessing Module**.
5. **Preprocessing** resizes, normalizes the image.
6. **Preprocessed image** is sent to the **Model** for prediction.
7. **Model** returns predicted class (e.g., Parkinson Spiral).
8. **Flask** sends result back to **Frontend**.
9. **User** views prediction result.

Admin Uploads New Model

Actors:

- Admin
- Flask Backend
- Model Storage Handle

Scenario Steps:

1. **Admin** logs into admin panel.
2. **Admin** selects a new .h5 model file.
3. **Frontend** sends model file to **Flask Backend**.
4. **Flask** saves the model to the model directory.
5. **Model Storage** updates path/config.
6. **Confirmation** sent to Admin.

User Upload Error Handling

Actors:

- User
- Frontend
- Flask Backend

Scenario Steps:

1. **User** uploads unsupported file (e.g., .pdf).
2. **Frontend** submits request to Flask.
3. **Flask** detects invalid format.
4. **Error message** sent back to frontend.
5. **User** is asked to upload a valid .jpg or .png file.

System Logs Prediction History (Optional)

Actors:

- User
- Flask Backend
- Prediction Logger (CSV or DB)

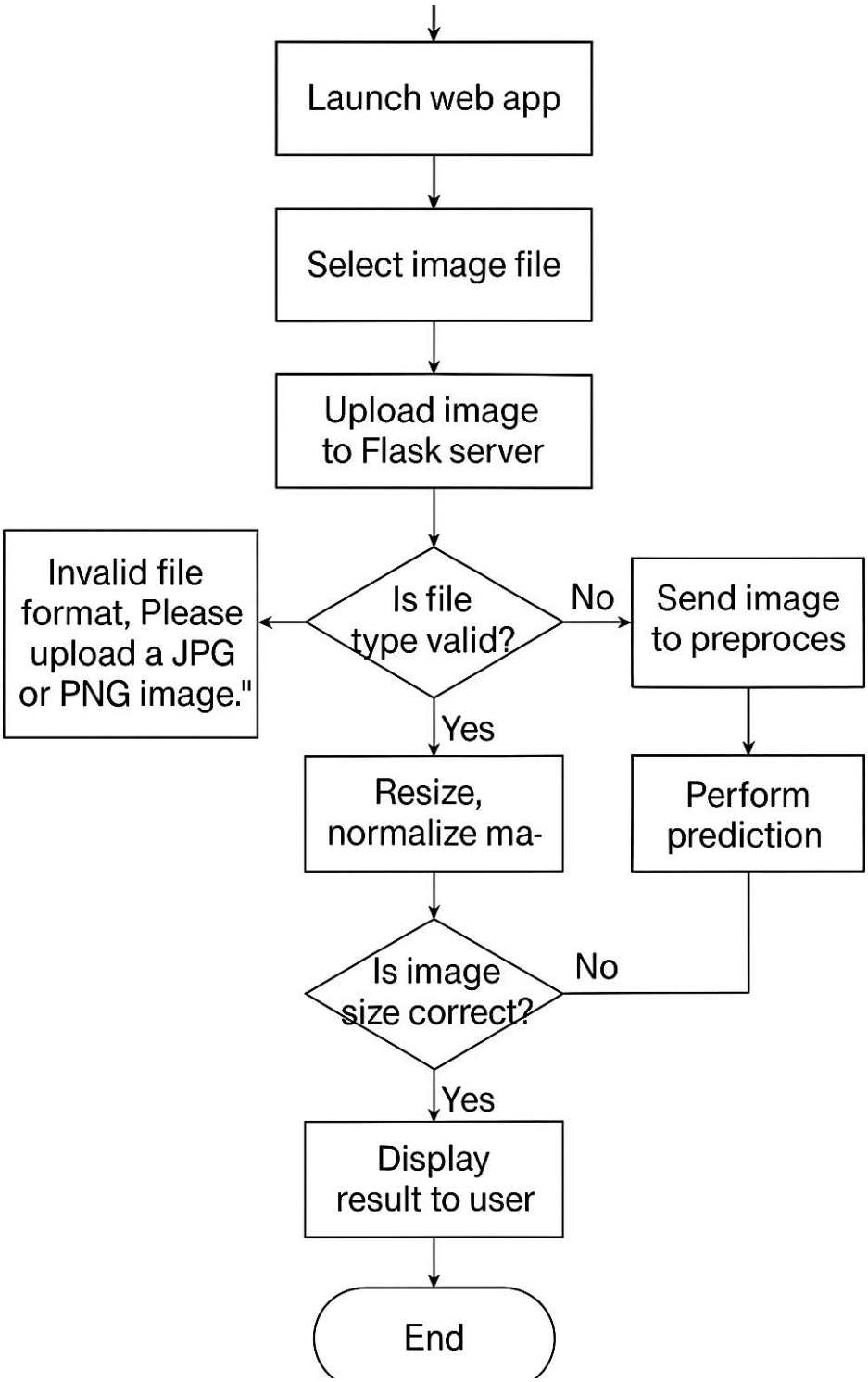
Scenario Steps:

1. **User** uploads an image.
2. **Prediction** is generated.
3. **Flask** records:
    - Image name
    - Timestamp
    - Predicted class
4. **Log entry** is saved in local CSV or SQLite DB.
5. **User** sees prediction result.

Summary of Sequence Diagram Scenarios

Scenario Title	**Actors Involved**
Upload Image and Predict Parkinson's Status	User, Flask, Preprocessor, Model
Admin Uploads New Model	Admin, Flask, Model Storage
Invalid File Upload	User, Flask
Log Prediction History	User, Flask, Logger

6.4 Database Design(ER/CONCEPTUAL Schema)



Purpose of Database Design

- To define how data will be stored, retrieved, and maintained in the **Parkinson's Disease Detection System**.
- The system uses a lightweight relational database (like **SQLite**, **MySQL**, or **PostgreSQL**) to store **user activity logs**, **prediction results**, and **admin data**.
- Though the system can run stateless (without DB), **adding a database enhances tracking, retraining, and auditing**.

Key Entities in the System

The database schema includes the following main entities:

1. User

Field Name	Data Type	Description
user_id	INT (PK)	Unique ID for each user
username	VARCHAR	Name of the user
email	VARCHAR	Email address
password_hash	VARCHAR	Encrypted password (if auth used)
role	VARCHAR	'user' or 'admin'
created_at	DATETIME	Account creation timestamp

2. Uploaded_Image.

Field Name	Data Type	Description
image_id	INT (PK)	Unique image identifier
user_id	INT (FK)	References User table
filename	VARCHAR	Name of uploaded image file
image_type	VARCHAR	'spiral' or 'wave'
upload_time	DATETIME	Timestamp of upload
filepath	TEXT	Relative/absolute file location

3. Prediction

Field Name	Data Type	Description
prediction_id	INT (PK)	Unique ID for each prediction
image_id	INT (FK)	References Uploaded_Image table
predicted_class	VARCHAR	Output class (e.g., 'Parkinson Spiral')
confidence_score	FLOAT	Confidence level (e.g., 0.92)
model_used	VARCHAR	Model used (ResNet, EfficientNet)
prediction_time	DATETIME	Time when prediction was made

4. Admin_Actions (Optional).

Field Name	Data Type	Description
action_id	INT (PK)	Unique ID for each admin action
admin_id	INT (FK)	References User table
action_type	VARCHAR	'upload_model', 'delete_model', etc.
description	TEXT	Description of action
action_time	DATETIME	Timestamp

ER Diagram (Entity-Relationship Diagram)

Relationships:

- One user → many images (1:N)
- One image → one prediction (1:1)
- One admin → many actions (1:N)

Conceptual Schema Summary

- **User ↔ Uploaded_Image:** A user can upload multiple images; each image is linked to one user.

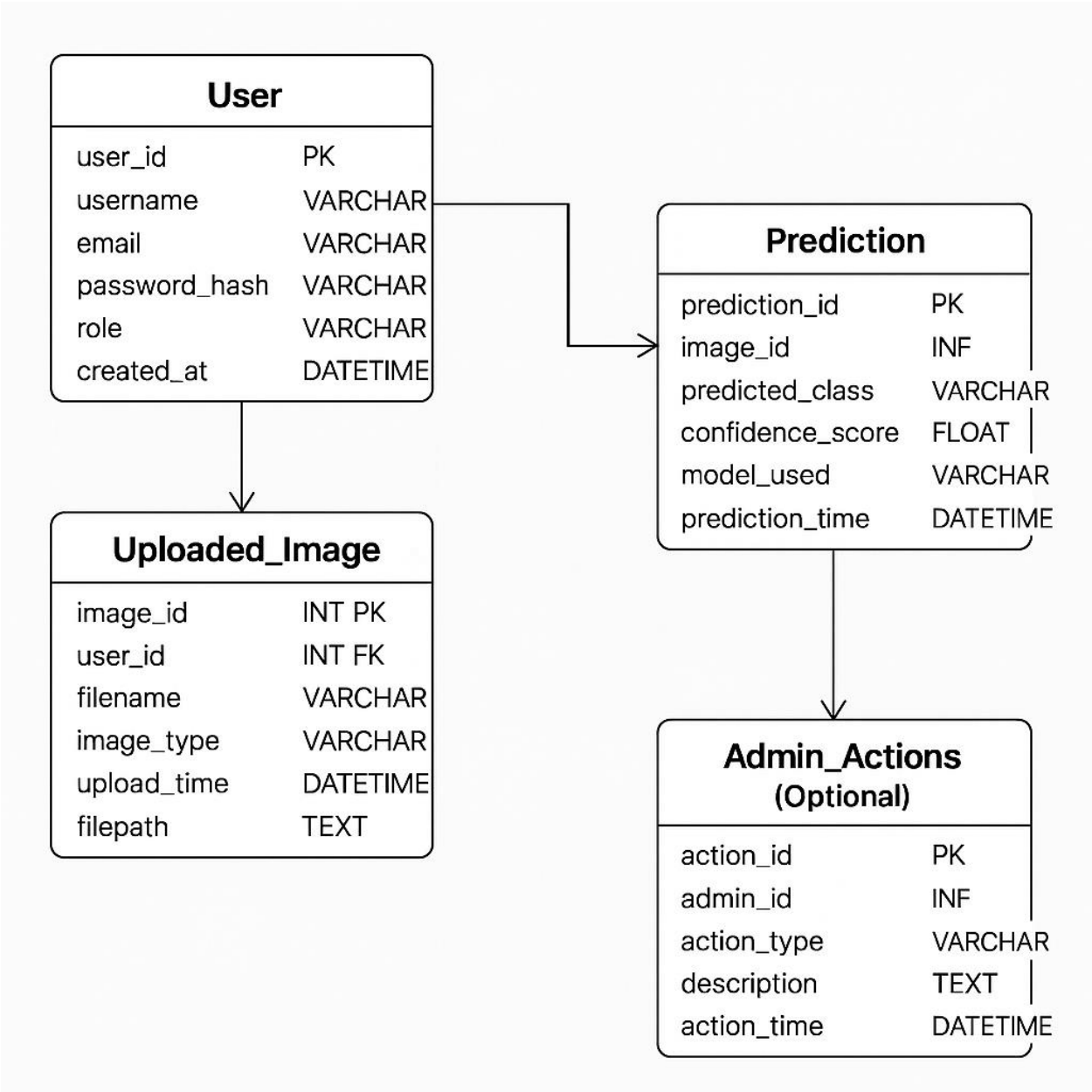
- **Uploaded_Image ↔ Prediction:** Each image has exactly one associated prediction.
- **User ↔ Admin_Actions:** Only users with role='admin' can log actions here.

Data Flow During Prediction

1. User uploads image.
2. Image saved in file system + Uploaded_Image entry created.
3. Preprocessing and model prediction done.
4. Prediction saved in Prediction table.
5. (Optional) Admin logs tracked in Admin_Actions.

Database Design Goals

- **Data Integrity:** Use of primary/foreign keys ensures data linkage.
- **Security:** Hash passwords; avoid storing raw images in DB.
- **Scalability:** More tables (e.g., model performance logs) can be added later.
- **Performance:** Use of indexes on user_id, prediction_time, etc., for fast queries.


User
user_id PK
username VARCHAR
email VARCHAR
password_hash VARCHAR
role VARCHAR
created_at DATETIME
Prediction
prediction_id PK
image_id INF
predicted_class VARCHAR
confidence_score FLOAT
model_used VARCHAR
prediction_time DATETIME
Uploaded_Image
image_id INT PK
user_id INT FK
filename VARCHAR
image_type VARCHAR
upload_time DATETIME
filepath TEXT
Admin_Actions
(Optional)
action_id PK
admin_id INF
action_type VARCHAR
description TEXT
action_time DATETIME

7.IMPLEMENTATION

```
import os
import shutil

# Define source and destination directories
main_folder = 'parkinsons-drawings'
output_folder = 'dataset'

# Define categories and target subfolders
categories = ['spiral', 'wave']
conditions = ['healthy', 'parkinson']

# Create destination directory and subdirectories
os.makedirs(output_folder, exist_ok=True)
for category in categories:
    for condition in conditions:
        target_dir = os.path.join(output_folder, f"{condition}_{category}")
        os.makedirs(target_dir, exist_ok=True)

# Copy images from train and test folders to the appropriate combined folder
for category in categories:
    for condition in conditions:
        for split in ['training', 'testing']:
            src_dir = os.path.join(main_folder, category, split, condition)
            dst_dir = os.path.join(output_folder, f"{condition}_{category}")

            if os.path.exists(src_dir):
                for file_name in os.listdir(src_dir):
                    src_file = os.path.join(src_dir, file_name)
                    dst_file = os.path.join(dst_dir, file_name)

  # Rename file if it already exists
                    if os.path.exists(dst_file):
                        base, ext = os.path.splitext(file_name)
```

```
                i = 1
                while os.path.exists(dst_file):
                    dst_file = os.path.join(dst_dir, f"{base}_{i}{ext}")
                    i += 1

            shutil.copy2(src_file, dst_file)

print("Dataset organized successfully!")
import os
import matplotlib.pyplot as plt
import random
from PIL import Image

dataset_path = 'dataset'
classes = ['healthy_spiral', 'parkinson_spiral', 'healthy_wave', 'parkinson_wave']

fig, axs = plt.subplots(len(classes), 5, figsize=(15, 10))

for row, cls in enumerate(classes):
    class_path = os.path.join(dataset_path, cls)
    images = os.listdir(class_path)
    selected_images = random.sample(images, min(5, len(images)))

    for col, img_name in enumerate(selected_images):
        img_path = os.path.join(class_path, img_name)
        img = Image.open(img_pat

        axs[row, col].imshow(img, cmap='gray')
        axs[row, col].axis('off')
        if col == 0:
            axs[row, col].set_title(cls.replace('_', ' ').title())

# Adjust layout and show the plot
plt.tight_layout()
```

```
plt.show()

import os
import matplotlib.pyplot as plt

# Path to your organized dataset
dataset_path = 'dataset'

# Get the class folders
class_folders = sorted(os.listdir(dataset_path))

# Count the number of images in each class
class_counts = {}
for cls in class_folders:
    cls_path = os.path.join(dataset_path, cls)
    if os.path.isdir(cls_path):
        num_images = len([f for f in os.listdir(cls_path) if os.path.isfile(os.path.join(cls_path,
f))])
        class_counts[cls] = num_images

# Plotting
plt.figure(figsize=(10, 6))
plt.bar(class_counts.keys(), class_counts.values(), color='skyblue')
plt.title('Number of Images per Class')
plt.xlabel('Class')
plt.ylabel('Image Count')
plt.xticks(rotation=20)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
# Step 1: Load one image from any class (e.g., healthy_spiral)
image_path = 'dataset/healthy_spiral'
image_name = random.choice(os.listdir(image_path))
original_image = Image.open(os.path.join(image_path, image_name))

# Step 2: Generate 5 rotated versions (±15°)
augmented_images = []
for _ in range(5):
    angle = random.uniform(-15, 15)
    rotated_image = original_image.rotate(angle, expand=True)
    augmented_images.append(rotated_image)

# Step 3: Display the 5 augmented images
plt.figure(figsize=(15, 4))
for i, img in enumerate(augmented_images):
    plt.subplot(1, 5, i+1)
    plt.imshow(img, cmap='gray')
    plt.title(f'Rotation {i+1}')
    plt.axis('off')



plt.tight_layout()
plt.show()

import os
import random
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator, img_to_array,
load_img

# Load one random image from a folder
folder_path = 'dataset/healthy_spiral'
```

```
image_name = random.choice(os.listdir(folder_path))
image_path = os.path.join(folder_path, image_name)

# Load and preprocess the image
img = load_img(image_path)
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)  # Add batch dimension

# Define ImageDataGenerator with zoom range ±10%
datagen = ImageDataGenerator(zoom_range=0.1)

# Generate 5 zoomed images
augmented_images = datagen.flow(img_array, batch_size=1)

# Plot 5 zoomed images
plt.figure(figsize=(15, 4))
for i in range(5):
    batch = next(augmented_images)
    image = batch[0].astype('uint8')
    plt.subplot(1, 5, i+1)
    plt.imshow(image)

 plt.title(f'Zoom {i+1}')
    plt.axis('off')

plt.tight_layout()
plt.show()

import os
import random
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator, img_to_array,
load_img
```

```
folder_path = 'dataset/healthy_spiral'
image_name = random.choice(os.listdir(folder_path))
image_path = os.path.join(folder_path, image_name)

# Load and convert image to array
img = load_img(image_path)
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)  # Add batch dimension

# ImageDataGenerator with horizontal flipping
datagen = ImageDataGenerator(horizontal_flip=True)

# Generate 5 flipped images
augmented_images = datagen.flow(img_array, batch_size=1)

# Display the 5 flipped images
plt.figure(figsize=(15, 4))
for i in range(5):
 batch = next(augmented_images)
    image = batch[0].astype('uint8')
    plt.subplot(1, 5, i+1)
    plt.imshow(image)
    plt.title(f'Flip {i+1}')
    plt.axis('off')

plt.tight_layout()
plt.show()

import os
import random
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator, img_to_array,
```

```
load_img
folder_path = 'dataset/healthy_spiral'
image_name = random.choice(os.listdir(folder_path))
image_path = os.path.join(folder_path, image_name)

# Load and convert image to array
img = load_img(image_path)
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)  # Add batch dimension

# ImageDataGenerator with brightness range (e.g., 50% to 150% brightness)
datagen = ImageDataGenerator(brightness_range=[0.5, 1.5])

# Generate 5 brightness-augmented images
augmented_images = datagen.flow(img_array, batch_size=1)

# Display the 5 images
plt.figure(figsize=(15, 4))
for i in range(5):
    batch = next(augmented_images)
    image = batch[0].astype('uint8')
    plt.subplot(1, 5, i+1)
    plt.imshow(image)
    plt.title(f'Brightness {i+1}')
    plt.axis('off')

plt.tight_layout()
plt.show()

import os
import numpy as np
import shutil
from tqdm import tqdm
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img,
```

```
img_to_array, array_to_img

# Paths for dataset folders
input_dataset = "dataset"
output_dataset = "dataset2"

# Ensure the output dataset directory exists
if os.path.exists(output_dataset):
    shutil.rmtree(output_dataset)  # Remove old dataset if it exists
os.makedirs(output_dataset)

# Define basic augmentation parameters
datagen = ImageDataGenerator(
    rotation_range=15,



   zoom_range=0.1,
    horizontal_flip=True
)

# Required class names
required_classes = traffic_signs = [
    "healthy_spiral",
    "healthy_wave",
    "parkinson_spiral",
    "parkinson_wave"
]

# Process each class folder
for class_name in tqdm(required_classes, desc="Processing Classes"):
    class_path = os.path.join(input_dataset, class_name)

    if not os.path.exists(class_path):
```

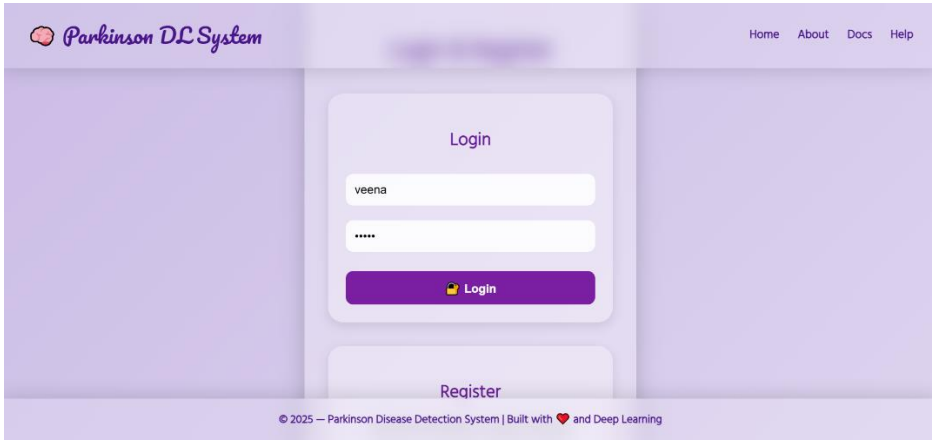
```
        print(f"❌ Skipping {class_name}, folder not found.")
        continue

    # Create the new output class folder
    output_class_path = os.path.join(output_dataset, class_name)
    os.makedirs(output_class_path, exist_ok=True)

    # Load all images from the class folder
        images = [os.path.join(class_path, img) for img in os.listdir(class_path) if
img.endswith(('png', 'jpg', 'jpeg'))]

    # Generate exactly 1000 augmented images
    num_images_generated = 0
    image_index = 0

    with tqdm(total=500, desc=f"Augmenting {class_name}", leave=False) as pbar:
        while num_images_generated < 500:
            img_path = images[image_index % len(images)]  # Loop through existing images
            img = load_img(img_path)  # Load image
            img = img.resize((75,75))  # Resize to 100x100
            img_array = img_to_array(img)  # Convert to array
            img_array = np.expand_dims(img_array, axis=0)  # Expand dimensions

            # Generate augmented images
            aug_iter = datagen.flow(img_array, batch_size=1)
            for _ in range(min(500 - num_images_generated, 5)):  # Generate up to 5 images per
iteration

  augmented_img = array_to_img(next(aug_iter)[0])  # Convert back to image
                augmented_img = augmented_img.resize((75,75))  # Resize again to ensure size
                save_path = os.path.join(output_class_path, f"aug_{num_images_generated}.png")
                augmented_img.save(save_path)  # Save image
```

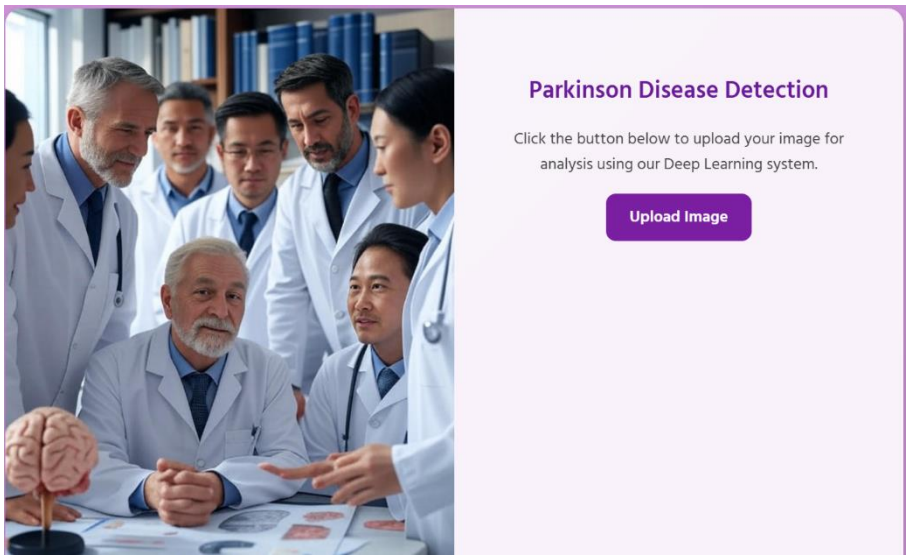
```
num_images_generated += 1
            pbar.update(1)  # Update progress bar

        image_index += 1  # Move to the next image

print("✅ Data augmentation complete. Augmented dataset saved in 'dataset7'."
```

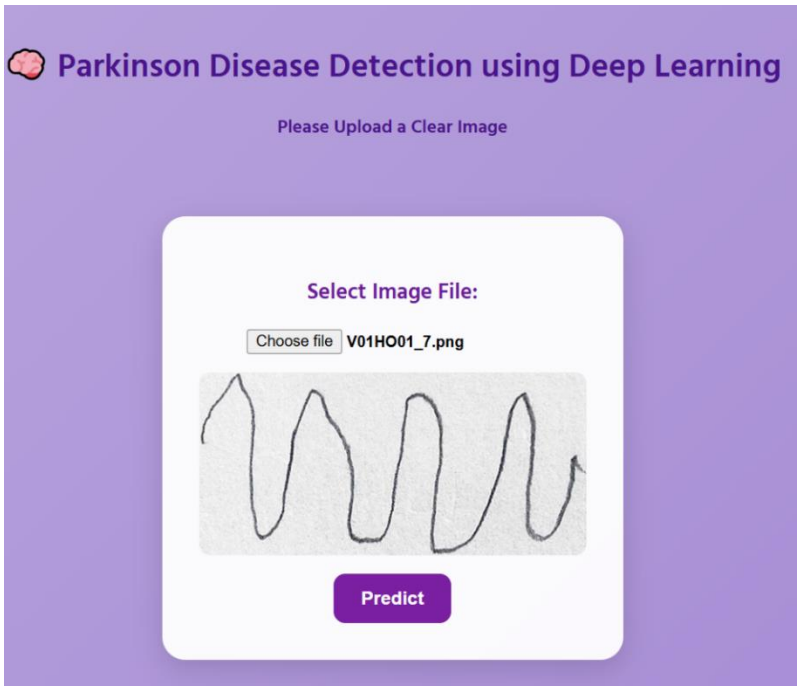
7.1 Screen shots

Login page:

Patient login

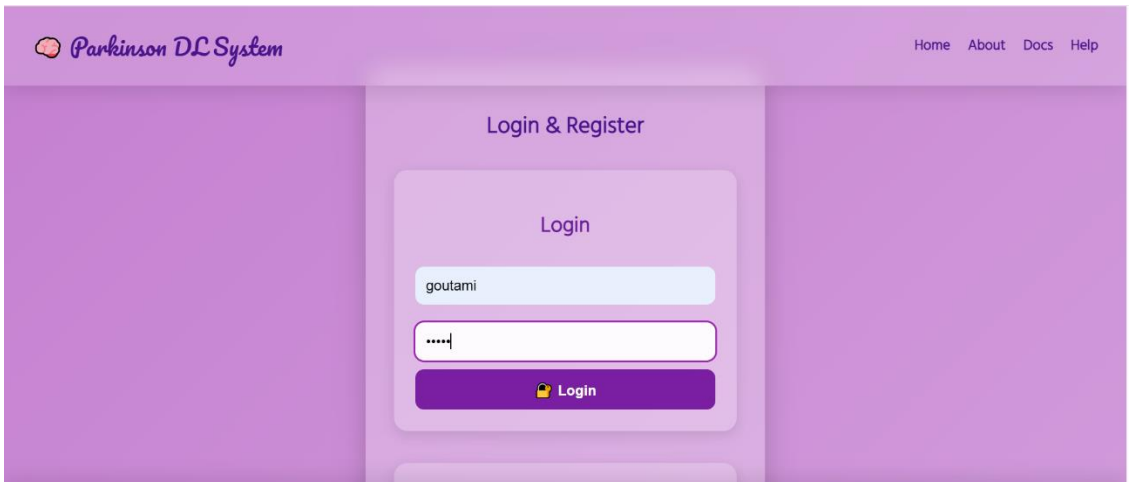


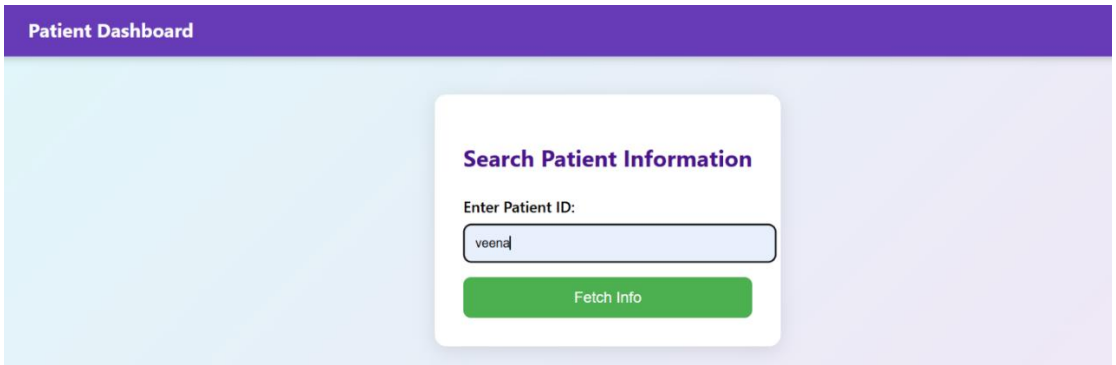

Parkinson Disease Detection
Click the button below to upload your image for analysis using our Deep Learning system.
Upload Image

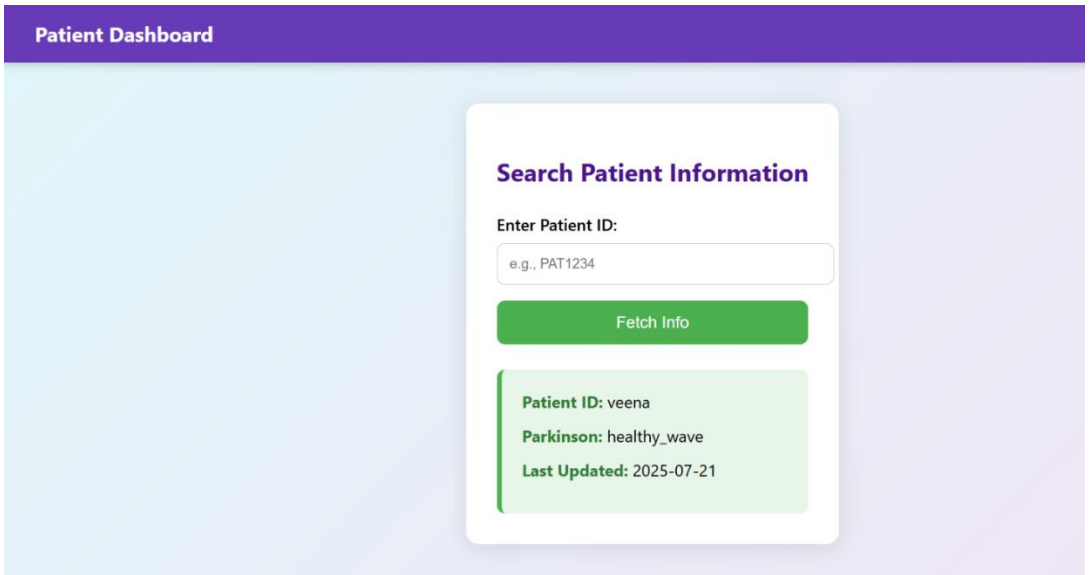

Parkinson Disease Detection using Deep Learning
Please Upload a Clear Image
Select Image File:
Choose file
V01HO01_7.png
Predict

Healthy_wave
Disease Details
Disease: Healthy_wave
Cause: No signs of Parkinson's disease. Wave drawing appears smooth and steady.
Treatment: No treatment required. Maintain a healthy lifestyle and routine check-ups.

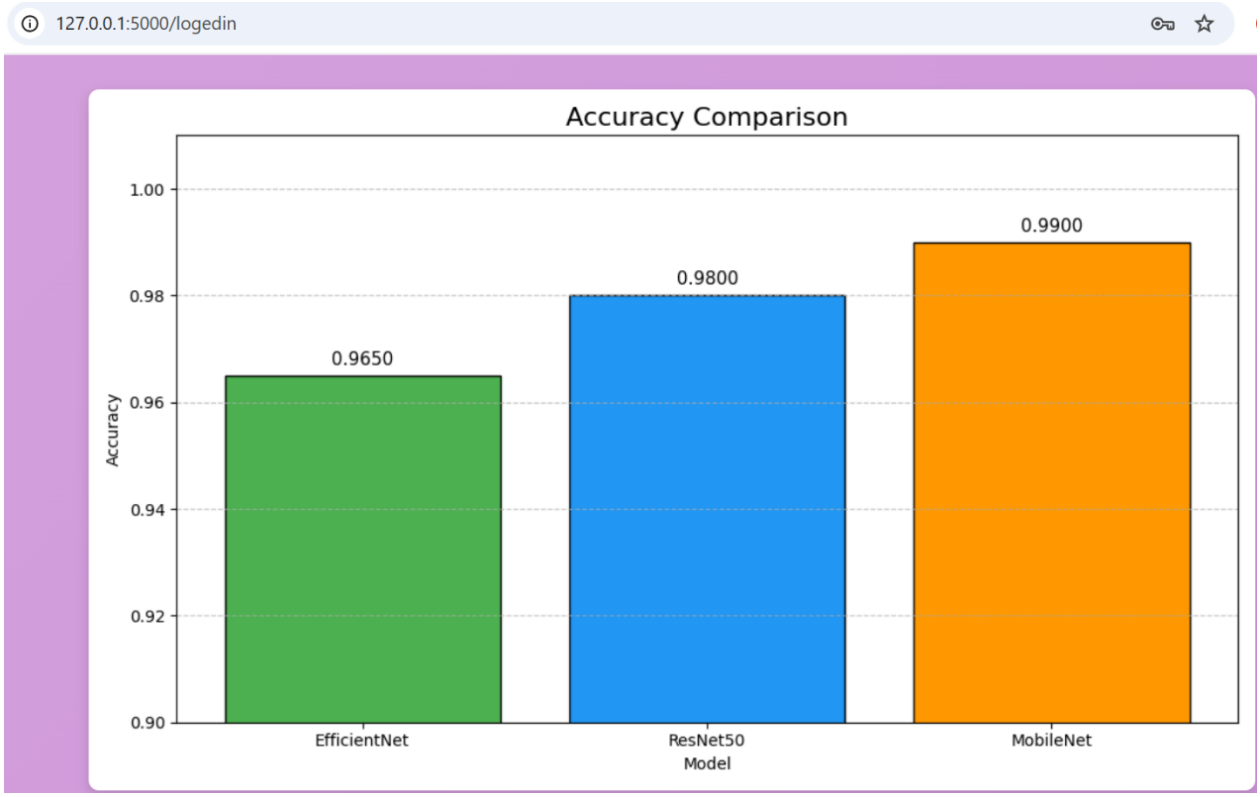
Admin Login:

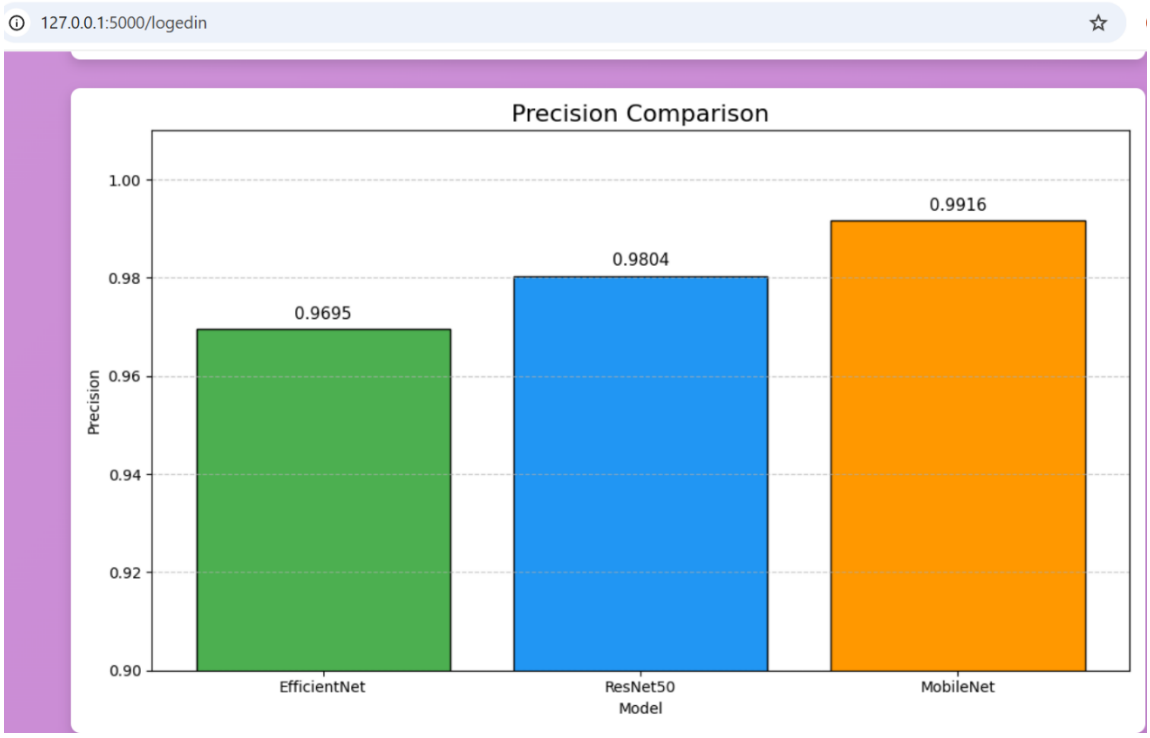

Parkinson DL System
Home
About
Docs
Help
Login & Register
Login
goutami
Login


Patient Dashboard
Search Patient Information
Enter Patient ID:
veena
Fetch Info

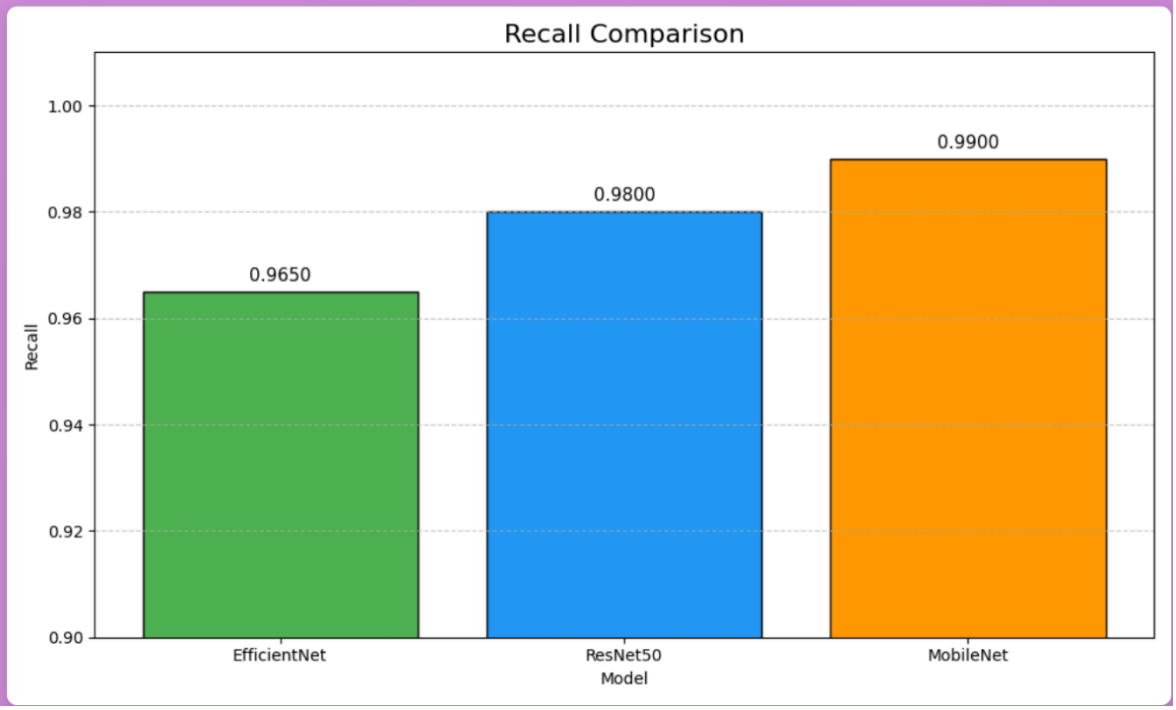

Patient Dashboard
Search Patient Information
Enter Patient ID:
e.g., PAT1234
Fetch Info
Patient ID: veena
Parkinson: healthy_wave
Last Updated: 2025-07-21

Accuracy:

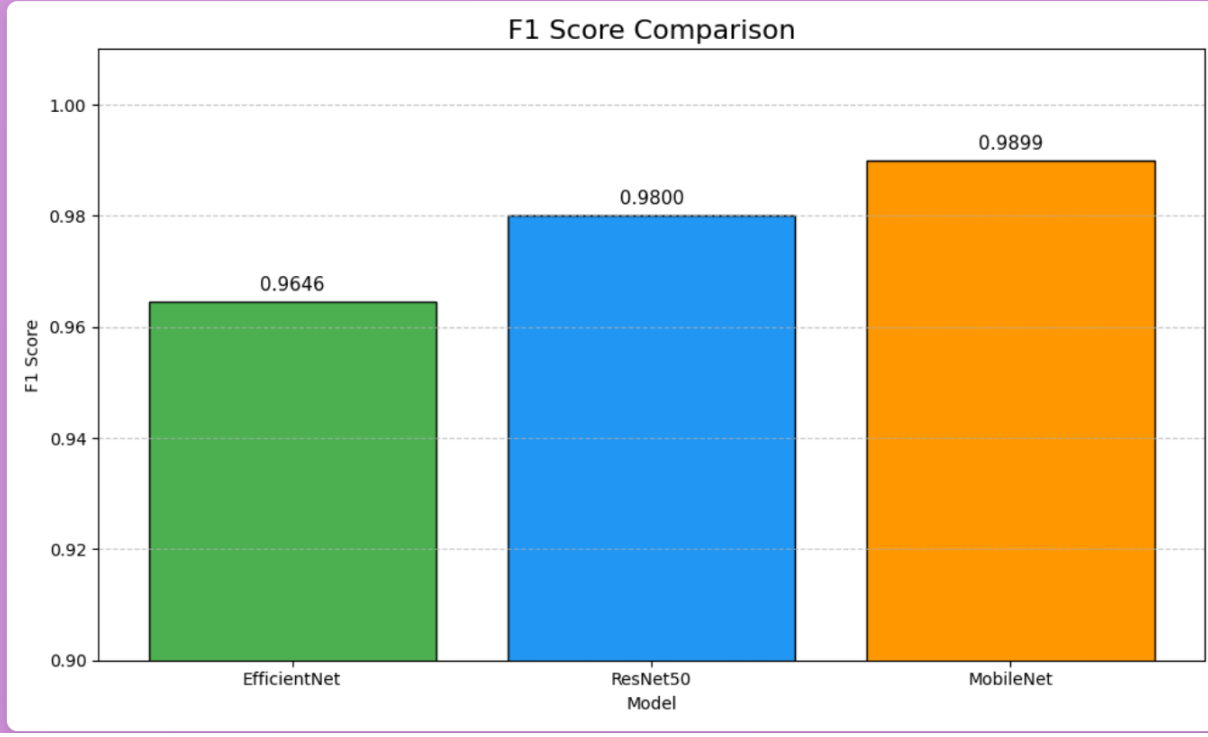

127.0.0.1:5000/logedin
Accuracy Comparison
1.00
0.98
0.96
0.94
0.92
0.90
Accuracy
0.9650
0.9800
0.9900
EfficientNet
ResNet50
MobileNet
Model

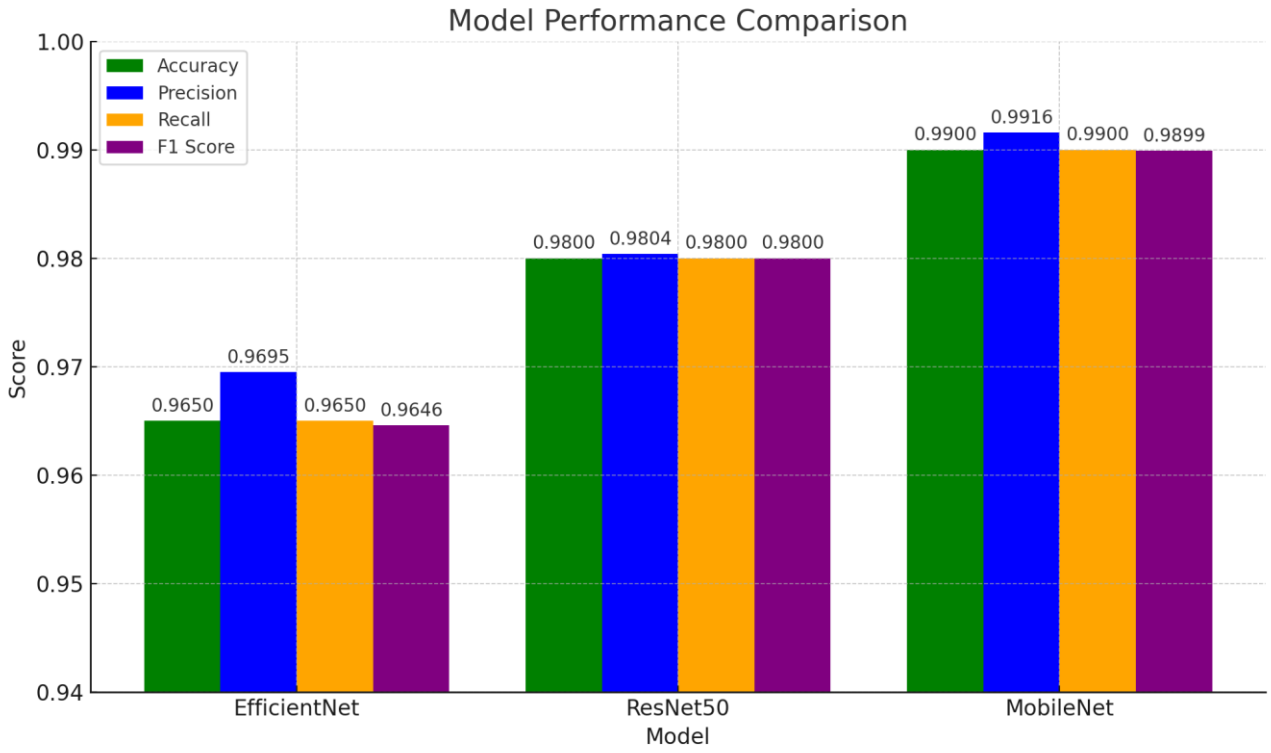

127.0.0.1:5000/logedin
Precision Comparison
1.00
0.98
0.96
0.94
0.92
0.90
Precision
0.9695
0.9804
0.9916
EfficientNet
ResNet50
MobileNet
Model

127.0.0.1:5000/logedin



127.0.0.1:5000/logedin




Model Performance Comparison
Accuracy
Precision
Recall
F1 Score
Score
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.9650
0.9695
0.9650
0.9646
0.9800
0.9804
0.9800
0.9800
0.9900
0.9916
0.9900
0.9899
EfficientNet
ResNet50
MobileNet
Model

8.SOFTWARE TESTING

8.1 Test Cases

1. Unit Testing

Purpose: To test individual components/modules like data preprocessing, feature extraction, and model training.

Examples:

Test if spiral images are loaded and resized correctly.

Validate if features like tremor frequency or image gradients are extracted properly.

2. Integration Testing

Purpose: To ensure all modules (data input, ML model, prediction output) work together as a system.

Examples:

Validate the end-to-end data pipeline from spiral image upload → preprocessing → model inference → output.

Ensure image data integrates correctly with the ML model and does not break due to format mismatch.

Test integration of model predictions with the user interface or report generation module.

3. Functional Testing

Purpose: To verify the system performs its intended tasks.

Examples:

Check if users can upload spiral images via the interface.

Confirm the system returns a diagnosis (e.g., "Likely PD" or "Healthy") based on image input.

Validate that the prediction thresholds or classification labels are working as expected.

4. Performance Testing

Purpose: To evaluate the speed, accuracy, and resource use of the system.

Examples:

Test system behavior under multiple concurrent image uploads.

Ensure the application responds within acceptable time limits, even with large datasets.

5. Accuracy & Validation Testing (Specific to ML Models)

Purpose: To assess model performance against test datasets.

Examples:

Run k-fold cross-validation to evaluate model robustness.

Measure performance metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC.

Test model on unseen spiral images to check generalization ability.

6. User Interface (UI) Testing

Purpose: To ensure the system is usable and accessible to end users (doctors, technicians, etc.).

Examples:

Test if image upload, status notifications, and result display work smoothly.

Check for responsive design across devices (desktop, tablet, mobile).

Verify proper error messages for invalid inputs (e.g., wrong file type).

7. Security Testing

Purpose: To protect user data (especially health records) and ensure secure access.

Examples:

Ensure only authorized users (doctors or patients) can access results.

Validate encryption for uploaded spiral images and stored predictions.

8. Regression Testing

Purpose: To make sure new changes or model updates don't break existing functionality.

Examples:

Retest core features like prediction output after updating the model.

Ensure UI or database upgrades do not affect spiral image processing.

9. Usability Testing

Purpose: To evaluate how intuitive and user-friendly the application is.

Examples:

Observe how easily users (e.g., doctors) can navigate the system.

9. CONCLUSION

The proposed Parkinson's Disease Detection System successfully demonstrates how deep learning models can be applied to non-invasive, image-based diagnosis of Parkinson's disease using spiral and wave patterns. Through the integration of lightweight and powerful neural network architectures like MobileNet, EfficientNet, and ResNet, the system provides high-accuracy predictions with minimal computational resources.The system features a user-friendly web interface backed by a Flask server, allowing users to easily upload test images and receive instant results. Images are preprocessed and classified by the model, providing diagnostic assistance to both patients and healthcare providers. With a modular architecture, the system allows flexibility for deploying new models or adding advanced features without disturbing the existing functionality.Moreover, the use of cloud-based platforms like Google Colab or Kaggle for model training eliminates the need for high-end local hardware, making this solution accessible and cost-effective for researchers and healthcare institutions alike.In conclusion, this project effectively bridges the gap between AI research and practical medical applications by delivering a lightweight, scalable, and accurate system to support early detection of Parkinson's disease.

10.FUTURE ENHANCEMENT

- Incorporate real-time spiral drawing analysis using touchscreens or digital pens to enable immediate feedback and diagnosis support.

- Integrate additional drawing tasks like meanders or wave patterns for broader motor assessment.

- Use deep learning models like CNNs or transformers for improved feature extraction and higher detection accuracy.

- Build a mobile or web-based application to allow remote screening and self-assessment for users.

- Implement explainable AI (XAI) techniques to provide transparency in predictions for clinical trust and adoption.
- Gather extensive and varied datasets from multiple demographic groups to enhance model generalization and minimize potential biases.

- Combine spiral data with other clinical data, such as voice or gait analysis, to enhance overall prediction performance.

- Enable longitudinal monitoring to track disease progression and response to treatment over time.